NETWORK PROGRAMMING · UTM

Laboratory Work 1: Concurrency in Practice

Authors: Pancenco Ina, Rusnac Nichita

Group: FAF-242

Teacher: Ernest Bîtca

Date: October 2026

---

1 Hardware and software

All measurements in this report were taken on a single laptop. Its configuration is listed in the table below, because the number and kind of processor cores, and the speed of the storage devices, directly determine how the different tasks behave.

Component	Value
CPU	AMD Ryzen 7 7435HS (8 identical cores, each with simultaneous multithreading)
Physical cores / logical processors	8 / 16
Clock	3.1 GHz base (as reported by `lab1 info`), up to 4.5 GHz boost
RAM	16 GB DDR5, one Samsung module (M425R2GA3PB0-CWMOD), running at 4 800 MT/s
System disk	NVMe SSD, Micron MTFDKBA1T0QFM, 1 TB
USB flash drive (Task 2)	Transcend JetFlash, 4 GB, FAT32; measured sequential read about 32 MB/s with an empty cache, which is the speed class of a USB 2.0 drive
USB port	AMD USB 3.10 eXtensible host controller, USB 3.0 root hub
Operating system	Windows 11 Pro, version 26H2 (build 26300)
Language and runtime	C# on .NET 9 (runtime 9.0.20, x64), Release build
Power	AC power, power plan “Turbo”

The processor shapes almost every result of Task 1. The Ryzen 7 7435HS has 8 physical cores, and every core can run two hardware threads at the same time (simultaneous multithreading, SMT, called Hyper-Threading by Intel). The operating system therefore sees 8 physical cores but 16 logical processors. The two hardware threads of one core are not two full cores: they share the core’s execution units and caches, and the second thread mainly uses the moments in which the first one waits, for example for memory. As lecture 1 explained, the second thread of a core therefore adds only a modest gain, in the order of 15 to 30 per cent, and nothing like a second core. All 8 cores are of the same kind and run at the same speed, so equal parts of work take about the same time on every core. The two numbers 8 and 16 are the reference lines in the charts of Task 1.

2 Method

The program. The whole laboratory work is implemented as a single C# console program. Every task and every configuration is selected by command-line parameters in the form `Labs.exe lab1 <command> [options]`; for example, Task 1a with eight threads is started with the command `lab1 1a` and the option for eight threads. The full list of commands is given in Appendix A.

The input data. The command `lab1 gen` writes the file `numbers.txt`, which contains 50 000 000 random integers between −1 000 000 and 1 000 000, one number per line. The file is 352 MB in

size. The random generator is started with a fixed seed (12345), so the same file can be produced again at any time. While the generator writes the file, it also counts how many of the numbers are negative, zero and positive, and stores these reference counts in a second file, `numbers.ref.txt`:

negative	zero	positive	total
25 007 776	22	24 992 202	50 000 000

For Task 2, the command `lab1 split` cuts `numbers.txt`, keeping the original order, into 8 files of 6 250 000 lines each, which are about 44 MB per file.

The correctness check. After every single run, the program compares the three counts it has computed with the reference counts and prints **OK** if they are identical or **WRONG** if they are not. A configuration only counts as a valid result if it reports **OK**. Every run in this report passed this check, with the single, intended exception of the unsynchronised version of Task 1b, whose purpose is precisely to show a wrong result.

Threads. All threads used by the tasks are dedicated operating-system threads that the program creates itself with `new Thread`, starts with `Start` and waits for with `Join`. The .NET thread pool is not used, so the number of threads in each experiment is exactly the number that was requested.

What is measured. The execution time is the wall-clock time of the actual work, measured with a `Stopwatch`. It does not include the start-up of the process, and in Task 1b it also does not include the loading of the numbers into memory, which the assignment explicitly excludes. The memory is reported as two high-water marks of the process: the peak working set, which is the largest amount of physical RAM the process occupied at any moment, and the peak private memory, which is the largest amount of memory the process had committed for itself. Because these values are maxima over the whole lifetime of a process, every configuration was run in a fresh process, so that one measurement cannot inherit the memory peak of another. Part of the working set belongs to the .NET runtime itself and is present even before any work starts.

How Task 1 was run. Before the measured runs, one unrecorded warm-up run read the whole of `numbers.txt`, so that the file was already held in the operating system's file cache. All measured runs of Task 1 therefore read the file from RAM, and they measure the CPU work of parsing and counting rather than the speed of the SSD. Every configuration was run three times, and the report uses the median of the three times. The repetitions formed the outer loop of the measuring script, so that a temporary slowdown of the laptop, for example a background task or the processor getting hot, would affect one repetition of every configuration rather than all three repetitions of a single one.

How Task 2 was run. Before every run of Task 2, the file cache of the USB drive was emptied by dismounting its volume with the script `dismount-volume.ps1`, which locks the volume and sends it the `FSCTL_DISMOUNT_VOLUME` request. When a volume is dismounted, Windows discards all of its cached data and mounts it again on the next access, so this has the same effect as ejecting, unplugging and replugging the drive, and all data really had to be read from the flash memory. A quick test confirmed it: reading one 44 MB part took 0.03 s while it was cached and 1.47 s right after a dismount. Each configuration of Task 2 was run three times, more than the single run the assignment requires, and the report uses the median.

Runtime settings. The program uses the workstation garbage collector in non-concurrent mode, so that the runtime does not start additional garbage-collection threads that would compete with the measured threads for the processor. Tiered compilation is switched off, so that all methods are compiled with full optimisation from the start and short runs are not partly measured in slower, unoptimised code.

3 Task 1: One file

3.1 Task 1a: each thread reads and parses its own part

In Task 1a, the file is divided into as many equal byte ranges as there are threads. Each thread opens the file on its own, reads only its own range through a private 1 MB buffer, and parses the digits directly from the bytes, without creating a string for each line. It counts the negative, zero and positive numbers in its own local counters, and the main thread adds the counts of all threads together after joining them.

The difficult part is the boundary between two neighbouring ranges, because a boundary computed from the file size will usually fall in the middle of a line. To solve this, both the start and the end of every range are moved forward to the beginning of the next line. Since two neighbouring parts apply the same rule to the same offset, the end of one part is always exactly the start of the next one. As a result every line belongs to exactly one part, so no line is counted twice and none is skipped.

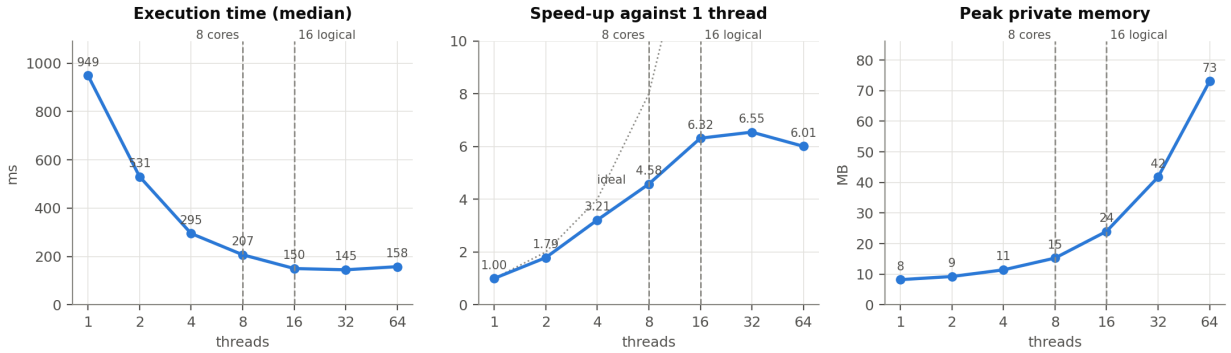


Figure 1: Task 1a: execution time, speed-up and peak private memory against the number of threads. The dashed vertical lines mark the 8 physical cores and the 16 logical processors.

Threads	1	2	4	8	16	32	64
Time, ms (median)	949	531	295	207	**150**	145	158
Speed-up	1.00	1.79	3.21	4.58	**6.32**	6.55	6.01
Efficiency (speed-up / threads)	100%	89%	80%	57%	40%	20%	9%
Peak private, MB	8	9	11	15	24	42	73
Peak working set, MB	26	27	29	33	42	58	90
Check	OK	OK	OK	OK	OK	OK	OK

3.2 Task 1b: three ways to count

In Task 1b, all 50 000 000 numbers are first loaded into an integer array, and this step is not measured. The array is then divided into 16 equal parts, one for each of the 16 logical processors, and 16 threads count the signs of their part. The three versions required by the assignment differ only in where the counters live and how they are protected. In the unsynchronised version, all threads increment the same three shared counters with a plain `counter++`. In the locked version, they use the same shared counters, but every single increment happens inside a `lock` statement. In the local version, each thread counts in its own local variables, and the 16 partial results are added together only after all threads have finished. As an additional fourth version, not required by the assignment, the shared counters were also incremented with the atomic operation `Interlocked.Increment`.

Version	Time (median of 3)	Totals	Peak memory
local (per-thread counters)	**77 ms**	correct	≈218 MB
unsync (shared, no synchronisation)	302 ms	**WRONG**	≈218 MB
interlocked (shared, atomic, extra)	1 057 ms	correct	≈218 MB
lock (shared, every increment locked)	10 261 ms	correct	≈218 MB

As the assignment requires, the unsynchronised version was run three times. Its totals are wrong in every run, and they are different every time. Across the three runs, only between 7.2 and 9.5 million of the 50 million numbers were counted, which means that between about 81 and 86 per cent of all increments were lost. The individual runs are:

Run	Time	negative	zero	positive	total	lost increments
1	329 ms	3 579 384	22	3 625 773	7 205 179	85.6%
2	291 ms	3 899 790	22	3 988 143	7 887 955	84.2%
3	303 ms	4 726 689	22	4 799 899	9 526 610	80.9%

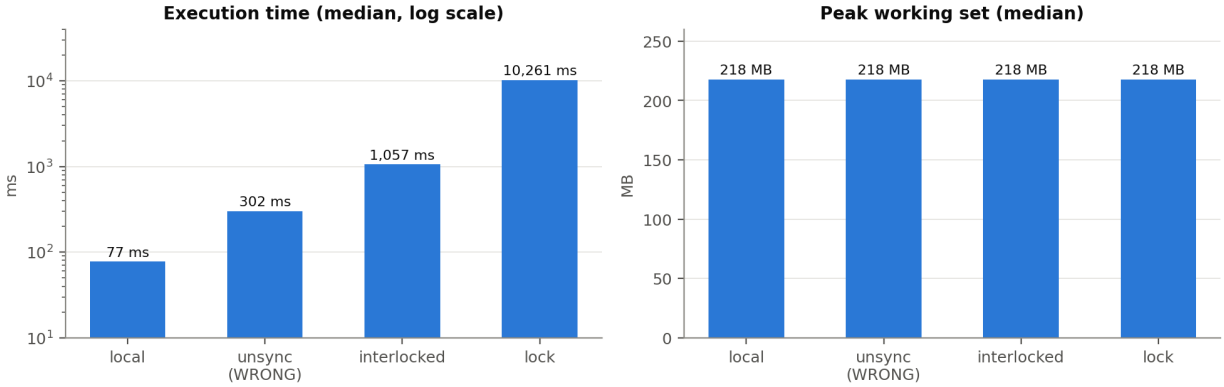


Figure 2: Task 1b: execution time and peak memory of each version, with 16 threads.

3.3 Task 1c: producer and consumers

Task 1c divides the work in a different way. A single thread, the reader, reads `numbers.txt` line by line with a `StreamReader` and collects the lines into batches of 10 000. Each full batch is added to a thread-safe `BlockingCollection` whose capacity is limited to 100 batches. When the queue is full, the call that adds a batch blocks the reader until a worker has taken a batch out. The workers repeatedly take the next batch through `GetConsumingEnumerable`, which blocks a worker while the queue is empty, and count the numbers of the batch in their own local counters. When the reader reaches the end of the file, it calls `CompleteAdding` to close the queue; the workers then finish as soon as the remaining batches have been taken, and their counts are added together.

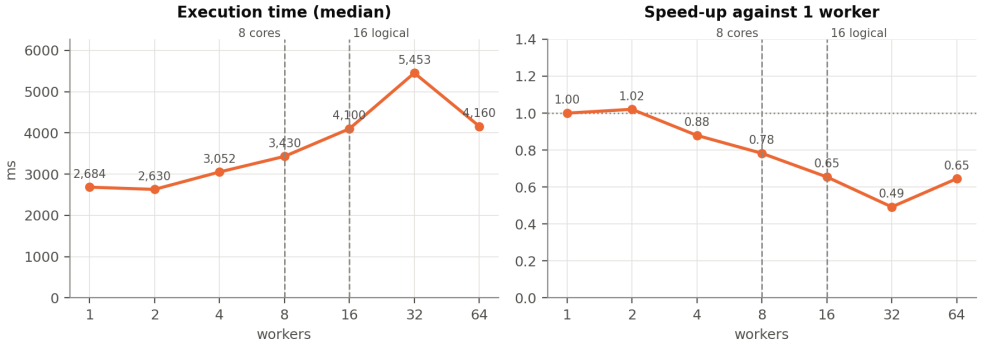


Figure 3: Task 1c: execution time and speed-up against the number of workers.

Workers	1	2	4	8	16	32	64
Time, ms (median)	2 684	**2 630**	3 052	3 430	4 100	5 453	4 160
Speed-up	1.00	1.02	0.88	0.78	0.65	0.49	0.65
Peak working set, MB	50	59	78	120	161	202	291

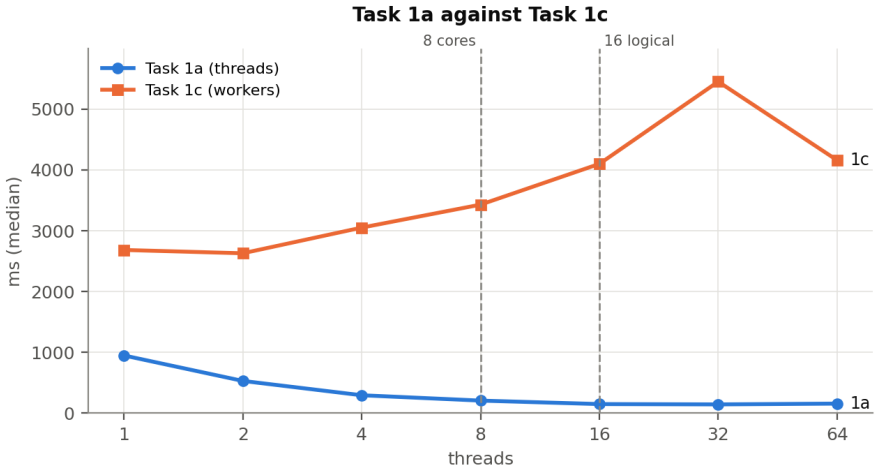


Figure 4: Execution time of Task 1a and Task 1c against the number of threads.

3.4 Answers to the questions of Task 1

1. At which number of threads does the speed-up stop growing? Relate it to your physical cores and logical processors.

In Task 1a, the speed-up keeps growing up to 16 threads, where it reaches 6.32×, and then it stops. With 32 threads it is 6.55× and with 64 threads 6.01×. These last differences are smaller than the normal variation between runs (the runs with 64 threads alone took between 156 and 238 ms), so the curve is simply flat after 16 threads. This is exactly the number of logical processors of the laptop: with 16 threads, every hardware thread of the processor has work to do, and there is no free hardware left.

The shape of the curve follows the structure of the processor. Up to 8 threads, every thread can have a physical core to itself, and the speed-up grows to 4.58×. The growth is not perfect, because the efficiency falls from 89% with 2 threads to 80% with 4 and 57% with 8. Between 8 and 16 threads, the threads have to share the cores through SMT, and the time falls only from 207 to 150 ms, a further 1.38× instead of 2×. This is the expected behaviour: the two threads of a core

share its execution units, so the second one only fills the gaps in which the first one waits (a parser has many such gaps, for example mispredicted branches and memory accesses). A gain of 38 per cent is somewhat higher than the 15 to 30 per cent mentioned in lecture 1, but of the same order.

The speed-up is therefore far below 16×, and the likely reasons are the following. Only 8 of the 16 hardware threads are full cores. When all cores are busy, the processor also runs them at a lower clock frequency than a single busy core, because of its power and temperature limits, so every thread becomes slower as more threads are added. All threads copy their data out of the same file cache, so they share the memory bandwidth and the last-level cache. Finally, the run takes only about 150 ms at its best, so fixed costs such as creating the threads, opening the file once per thread and the sequential steps at the beginning and at the end are no longer negligible. This last effect is described by Amdahl's law.

2. Why do 32 and 64 threads not help? Are they slower, and why?

Once there are more threads than the 16 logical processors, there is no additional hardware on which the extra threads could run. The operating system can only take turns, letting each thread run for a short time slice before switching to another one. The total amount of CPU work stays exactly the same, and on top of it come the costs of switching between threads and of refilling the processor caches after each switch, so more threads than logical processors can never make a CPU-bound task faster.

In Task 1a, 32 and 64 threads are nevertheless hardly different from 16 (145 and 158 ms against 150 ms). The threads of Task 1a share no data, and even with 64 threads every part of the file is still about 5.5 MB large, so the time lost to switching is small compared with the useful work. The 64-thread configuration is the slowest of the three, but the difference is within the variation between runs (156 to 238 ms for 64 threads alone), so it should not be over-interpreted. The extra threads still cost memory, as the next question shows, and they bring no gain at all. In Task 1c, where the threads compete for a shared queue, more threads are clearly slower, which is discussed in question 5.

3. Does the peak memory depend on the number of threads? Why?

The peak memory depends on the number of threads whenever each thread owns some data of its own, and it does not depend on it when the data is shared. Task 1a shows the first case very clearly. Its peak private memory grows from 8 MB with one thread to 73 MB with 64 threads, which is about 1 MB for every additional thread. That matches the 1 MB read buffer that each thread allocates, plus the small part of the thread's own stack that is actually used.

To confirm that the read buffer is responsible, Task 1a with 64 threads was run once more with a buffer of 64 KB per thread instead of 1 MB, together with the normal 1 MB configuration in the same session (three runs of each, medians):

```
.\bin\Release\net9.0\Labs.exe lab1 1a --threads 64                  # 1 MB buffer per
    thread (default)
.\bin\Release\net9.0\Labs.exe lab1 1a --threads 64 --buffer-kb 64   # 64 KB buffer per
    thread
```

Task 1a, 64 threads	Time (median of 3)	Peak working set	Peak private	Check
1 MB buffer per thread	99 ms	91 MB	74 MB	OK
64 KB buffer per thread	88 ms	31 MB	15 MB	OK

With the smaller buffers, the peak private memory fell from 74 MB to 15 MB and the peak working set from 91 MB to 31 MB, which is only about 6 MB above the 25 MB that the .NET runtime occupies before any work starts. The saving of about 60 MB is exactly what the buffers predict:

$64 \times (1024 - 64)$ KB = 60 MB. The run did not become slower (88 ms against 99 ms, a difference within the variation between runs), because a 64 KB read is still large enough that the fixed cost of each read call is negligible compared with parsing 64 KB of numbers. The private memory per thread is now about 100 KB, the 64 KB buffer plus the used part of the stack. These two configurations were measured later than the main table of Task 1a, with the laptop in a faster state, so their times should only be compared with each other.

Task 1c also uses more memory with more workers, growing from about 50 MB to 291 MB in the largest configuration. Here the memory is mostly made of the strings that the reader creates. Each worker holds a batch of 10 000 strings while it counts it, so with more workers more batches are in use at the same time. When the garbage collector runs, these batches are still alive, so they survive the collection and are moved to an older generation of the heap, which is cleaned up only rarely.

Task 1b shows the opposite case. Its memory is the same in every version, about 218 MB, because it is dominated by the shared array of 50 million integers (200 MB), while the counters add only a few bytes per thread. Task 2 again grows with the number of threads, because each thread holds a whole 44 MB file in memory, as described in Section 4. In summary, memory grows with the number of threads only for data that every thread owns privately, such as buffers and batches, while shared, read-only data costs the same regardless of how many threads read it.

4. Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.

Ordered from the fastest to the slowest, the versions are local (77 ms), unsynchronised (302 ms), interlocked (1 057 ms) and locked (10 261 ms). The local, interlocked and locked versions produce the correct totals. The unsynchronised version does not.

The unsynchronised version is wrong because an increment such as `shared.Positive++` is not a single step. The processor has to read the current value from memory, add one to it, and write the result back. If two threads read the same old value at nearly the same time, both add one and both write back the same new value, so one of the two increments is simply lost. With 16 threads constantly incrementing the same counters, this happened for between about 81 and 86 per cent of all increments, and because it depends on the exact timing of the threads, the result was different in every run (between 7.2 and 9.5 million counted instead of 50 million). The count of zeros was correct every time, but only by luck: there are only 22 zeros among 50 million numbers, so two threads almost never increment that counter at the same moment. Interestingly, the unsynchronised version is also about 4 times slower than the local one, even though it does no synchronisation at all. The three shared counters lie in the same 64-byte cache line, and each write by one core invalidates that line in the caches of the other cores, so the line is constantly passed back and forth between the cores.

The locked version is correct, but it is the slowest by far, about 130 times slower than the local one. Each of the 50 million increments has to acquire and release the lock, so only one thread can make progress at any moment, while the other 15 wait. The work inside the lock is therefore performed one thread at a time again, exactly as lecture 2 described the lock as a queue, and on top of that every hand-over of the lock between threads costs time, about 205 ns of wall-clock time per number.

The interlocked version is also correct and is about 14 times slower than the local one. `Interlocked.Increment` makes the read, the addition and the write a single atomic hardware operation, so no update can be lost and no thread is ever blocked. Each atomic increment still needs exclusive ownership of the cache line that holds the counter, however, so the cores still have to take turns, only at a much lower cost than with a lock (about 21 ns per number).

The local version is both correct and the fastest. Each thread counts in its own local variables, which the compiler keeps in processor registers, so there is no shared state while counting, and therefore neither a race nor any waiting. Each thread writes its result only once, at the very end, and the main thread adds the 16 results after `Join` has guaranteed that all threads are finished.

This is the first step of the ladder of fixes from lecture 2, "don't share", and the measurement confirms its main message: it is correct and at the same time even faster than the broken version.

5. Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?

Task 1a is faster than Task 1c for every number of threads. With one thread, Task 1a needs 949 ms and Task 1c 2 684 ms, and at their best configurations Task 1a needs 145 ms while Task 1c needs 2 630 ms, which makes Task 1a about 18 times faster.

The main reason is that Task 1c contains a stage that cannot be parallelised. A single reader thread has to read and split every one of the 50 million lines, so no matter how many workers parse the numbers, the whole run can never be shorter than the time the reader alone needs, which is about 2.6 s. This is an example of Amdahl's law: the sequential part of a program limits its total speed-up. In Task 1a, by contrast, both the reading and the parsing are divided among all threads. In addition, the reader of Task 1c does more work for each line than Task 1a does. It converts the bytes into characters and creates a separate string object for each line, which means 50 million objects that the garbage collector afterwards has to clean up, and the workers then parse these strings with `int.Parse`. Task 1a reads the digits directly from the byte buffer and creates no objects at all. Passing the work through the queue also has a cost, although it is small with batches of 10 000 lines, because there are only about 5 000 queue operations in total.

Task 1c stops getting faster immediately. With one worker it takes 2 684 ms and with two workers 2 630 ms, an improvement of only 2 per cent. A single worker already keeps up with the reader almost completely, so any additional workers mostly wait on an empty queue. From four workers on, the task becomes slower, reaching 3 052 ms with 4 workers and more than 4 000 ms with 16 or more. More workers compete for the internal lock of the queue, more threads have to be put to sleep and woken up again, and the garbage collector has more threads to stop every time it runs. The values for 32 and 64 workers (5 453 and 4 160 ms) are not in the expected order, which is measurement noise: the runs with 32 workers alone ranged from about 4.8 to 5.8 s.

6. What would happen with batches of a single line? And with a queue that has no size limit?

Both variants were measured with 1 and with 8 workers, together with the normal configuration of the assignment in the same session, so that the three rows can be compared directly (three runs of each, medians; every run reported **OK**). The commands were:

```
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 8                  # the assignment:
    batches of 10000, at most 100
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 8 --batch 1        # batches of a
    single line
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 8 --capacity 0     # a queue with no
    size limit
```

and the same three commands with `--workers 1`.

Variant	1 worker	8 workers	Peak working set (1 / 8 workers)
batches of 10 000 lines, at most 100 batches (the assignment)	1 359 ms	1 724 ms	51 / 120 MB
batches of 1 line, at most 100 batches	14 841 ms	43 959 ms	34 / 34 MB
batches of 10 000 lines, **no size limit**	1 353 ms	1 764 ms	50 / 120 MB

These runs were made later than the main table of Task 1c, with the laptop in a faster state (the normal configuration took 1.36 and 1.72 s instead of 2.68 and 3.43 s), so they should be compared with each other and not with that table.

With batches of a single line, every number becomes a separate queue operation, so the reader and the workers have to take the queue's internal lock and signal one another 50 million times instead of about 5 000 times. This coordination costs far more than counting a single number, and the measurement shows it clearly. With one worker, the run took 14.8 s, about 11 times longer than with batches of 10 000 lines. With eight workers it took 44.0 s (the three runs ranged from 38.2 to 47.6 s), about 26 times longer than the normal configuration and three times longer than with a single worker. More workers therefore make it worse, because all of them fight over the same lock for every single number and are repeatedly put to sleep and woken up again whenever the queue runs empty. With eight workers, each number cost about 0.9 µs of wall-clock time, far more than the parsing and counting itself. The memory, on the other hand, was the lowest of all Task 1c runs: a peak working set of 34 MB with any number of workers, only about 9 MB above what the runtime occupies on its own, because the queue can never hold more than 100 lines and no worker holds a batch of 10 000 strings. The experiment shows the purpose of batching: each queue operation has a fixed cost, and a batch must contain enough work to make that cost negligible.

A queue without a size limit removes the mechanism that slows the reader down when the workers cannot keep up. The reader never has to wait, and this missing feedback is called a lack of backpressure. On this laptop the change made no measurable difference: 1.35 s against 1.36 s with one worker and 1.76 s against 1.72 s with eight, which is within the variation between runs, and the peak memory was the same (50 and 120 MB). This confirms that the reader is the slow side: the workers empty the queue faster than the reader fills it, so the queue never comes close to 100 batches, and the limit never has to block the reader. If the workers were slower, however, for example because the work per number were heavier, the queue would keep growing until it held the entire file as strings. That would be roughly 50 million strings of 40 to 48 bytes each, or about 2 to 2.4 GB, compared with the roughly 48 MB that 100 batches can hold. The size limit of the queue is therefore what keeps the memory bounded: it forces the reader to adapt to the speed of the workers, and not the other way round.

4 Task 2: Many files on slow storage

In Task 2, a whole file is the unit of work. A thread reads a file in a single operation with `File.ReadAllBytes`, which places the entire file in one byte buffer, and then parses the buffer with the same parser that Task 1a uses. The 8 files are dealt to the threads in turn (file i goes to thread $i \bmod T$), so that each thread receives the same number of files, $8/T$. Besides the total time, each thread also measures how long its reading and its parsing take, which makes it possible to see where the time is spent. The files are read only from the USB flash drive, whose volume was dismounted before every run so that nothing came from the file cache. An earlier attempt without this step read all 8 files in only 468 ms, about 800 MB/s, which is far too fast for a flash drive: that data came from the cache, and the run is not used in this report. The runs below were started with the following commands, each one preceded by a dismount (three runs of each, medians):

```
.\lab1\scripts\dismount-volume.ps1 -Drive D
.\bin\Release\net9.0\Labs.exe lab1 2 --dir D:\lab1 --threads 1 --label usb-cold --csv
    lab1\results\task2.csv
# the same with --threads 2 and --threads 4
```

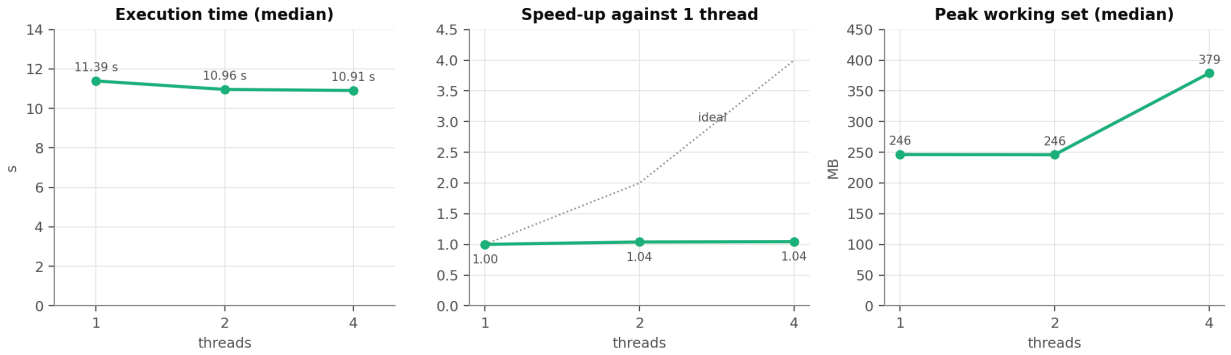


Figure 5: Task 2: execution time, speed-up and peak working set against the number of threads, with the files on the USB drive and an empty cache.

Threads	1	2	4
Time (median of 3)	11 391 ms	10 960 ms	10 908 ms
Speed-up	1.00	1.04	1.04
Reading, summed over threads	10 959 ms	20 118 ms	35 137 ms
Parsing, summed over threads	428 ms	425 ms	423 ms
Peak working set	246 MB	246 MB	379 MB
Peak private	228 MB	228 MB	361 MB
Check	OK	OK	OK

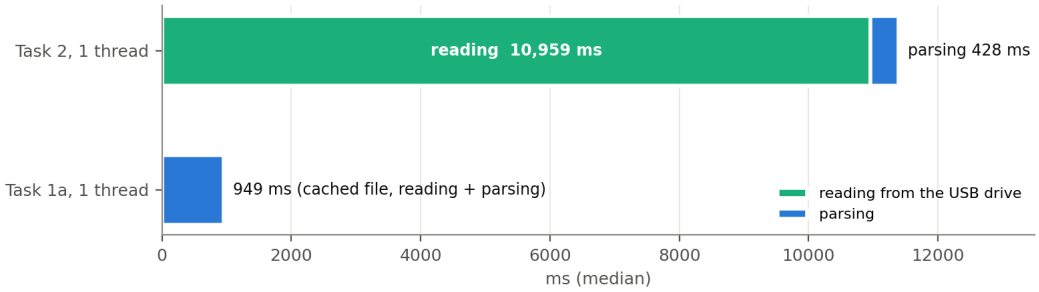


Figure 6: Where the time goes with one thread: Task 2 reading from the USB drive, compared with Task 1a reading the cached file.

4.1 Answers to the questions of Task 2

1. Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound?

Task 1a processed exactly the same numbers with one thread in 949 ms, reading from the file cache. Task 2 with one thread took 11 391 ms, so it took about 12 times longer. The time measured inside each thread shows where the difference comes from: reading the 8 files took 10 959 ms, which is 96% of the total and corresponds to about 32 MB/s (352 MB in 10.96 s), while parsing the numbers took only 428 ms. Parsing is the same CPU work as in Task 1a, and it is even shorter here because the whole file is already in one buffer and no time is spent copying it out of the cache. For 96% of the run the processor is idle and waits for the drive, so on the USB drive the work is clearly I/O-bound. A faster processor could save at most the 4% parsing share, whereas a faster drive would shorten the run almost in proportion to its speed.

2. On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?

On the flash drive, more threads hardly help: 2 threads gave a speed-up of 1.04 (10 960 ms) and 4 threads also 1.04 (10 908 ms). A single flash drive connected through a single USB link delivers a fixed amount of data per second, however many requests it receives at the same time. Parallel reads simply share this bandwidth between them, which the measurements show directly: the reading time summed over all threads grows from 11.0 s with one thread to 20.1 s with two and 35.1 s with four, because every thread now spends most of its time waiting while the drive serves the other threads, whereas the total time stays the same. The only part of the work that can overlap is the parsing, whose share of the total time is p (so the reading share is $1 - p$). Amdahl's law limits the speed-up with T threads to

$$S(T) = \frac{1}{(1-p) + p/T}, \qquad S(\infty) = \frac{1}{1-p}.$$

With the measured $p = 428/11\,391 \approx 0.038$, this gives $S(2) \approx 1.02$, $S(4) \approx 1.03$ and an absolute limit of $S(\infty) \approx 1.04$, however many threads are used. The measurement reaches this limit already with two threads, because while one thread parses a file, another one is already reading the next one, so the parsing is completely hidden behind the reading. The run with two threads (10.96 s) lasts exactly as long as the pure reading with one thread (10.96 s): the drive was busy all the time, and nothing more can be gained.

Lecture 1 is nevertheless right that concurrency helps I/O-bound work, but only for a particular kind of waiting. When a program waits for a server on the network, most of the waiting is latency: the request travels to the server, the remote machine does its work, and the answer travels back, while the local network connection is almost idle the whole time. Ten such requests can wait at the same time, so a concurrent program overlaps their waiting periods, and the total time drops almost tenfold. Concurrency helps when the waiting is spent on someone else's capacity, which is called latency-bound I/O. It does not help when the waiting is caused by the limited bandwidth of the device itself, which is called throughput-bound I/O, as with a single USB stick: a resource that is already fully busy cannot deliver data any faster just because more threads are asking for it.

Memory. The peak memory of Task 2 also grows with the number of threads (a peak working set of 246 MB with one and two threads and 379 MB with four), because every thread holds a complete 44 MB file in a single `byte[]`. Even with one thread, the peak private memory of 228 MB corresponds to about five files, and with four threads 361 MB corresponds to all eight. Arrays of this size are placed on the large object heap of .NET, which is cleaned up only during the rare full (generation 2) garbage collections, so several buffers of files that have already been processed remain in memory until the next such collection runs.

5 Conclusion

The measurements show that on this laptop Task 1 is CPU-bound in all three of its forms, while Task 2 is I/O-bound. The table summarises the evidence for each part.

Work	Limited by	Evidence from the measurements
Task 1a (parsing a cached file)	the CPU	The time falls as threads are added, up to 6.32× at 16 threads, and stops improving at the 16 logical processors (8 physical cores with SMT).
Task 1b (counting an in-memory array)	the CPU and the synchronisation	There is no I/O at all. The time depends only on how the threads share the counters: 77 ms with local counters against 10 261 ms with a lock.
Task 1c (one reader, N workers)	the CPU of a single thread	The file is in the RAM cache (Task 1a reads it in 145 ms at best), yet Task 1c needs about 2.6 s with any number of workers, because the single reader, which creates 50 million strings, sets the pace.
Task 2 (8 files on a USB flash drive)	the bandwidth of the drive	96% of the time is spent reading, at about 32 MB/s. Two and four threads give a speed-up of only 1.04, while the reading time summed over the threads grows from 11.0 to 35.1 s.

The two kinds of work can be told apart from the measurements alone. CPU-bound work becomes faster when threads are added, until the processor runs out of hardware threads, which on this machine happens at 16 threads, and during the whole run the processor is busy. I/O-bound work leaves the processor mostly idle, and its duration is set by the device it waits for. Additional threads help I/O-bound work only if the device can serve several requests in parallel, which a single USB flash drive cannot do.

The laboratory work also illustrates a lesson about correctness. Shared counters without synchronisation are fast but wrong, and protecting them with a lock makes them correct but 130 times slower. The best design is not to share the counters at all, but to let every thread count on its own and to add the results after `Join`, which is both correct and the fastest of all versions. The correctness check proved its value here: it detected the race condition in the unsynchronised version of Task 1b, which still produced plausible-looking numbers that were simply wrong.

A Building and running the program

The program requires the .NET 9 SDK, Windows PowerShell 5.1 or newer for the measuring scripts, and about 1 GB of free disk space for the generated data. The commands below are run in PowerShell; the options of the program are written exactly as the program expects them.

```
# from the repository root (the folder with Labs.csproj)
dotnet build Labs.csproj -c Release
.\bin\Release\net9.0\Labs.exe lab1 info          # hardware report
.\bin\Release\net9.0\Labs.exe lab1 gen           # numbers.txt and the reference counts
.\bin\Release\net9.0\Labs.exe lab1 split         # 8 parts for Task 2

# single runs
.\bin\Release\net9.0\Labs.exe lab1 1a --threads 8
.\bin\Release\net9.0\Labs.exe lab1 1b --mode unsync      # unsync, lock, local or
    interlocked
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 4        # batches of 10000, at most
    100 in the queue
.\bin\Release\net9.0\Labs.exe lab1 2 --dir E:\lab1 --threads 2

# additional experiments (Task 1, questions 3 and 6)
.\bin\Release\net9.0\Labs.exe lab1 1a --threads 64 --buffer-kb 64   # 64 KB buffer per
    thread instead of 1 MB
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 8 --batch 1         # batches of a
    single line
```

```
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 8 --capacity 0      # a queue with no
    size limit
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 1 --batch 1         # the same two
    variants with one worker
.\bin\Release\net9.0\Labs.exe lab1 1c --workers 1 --capacity 0

# all measurements (from the lab1\scripts folder)
Set-ExecutionPolicy -Scope Process Bypass
.\run-task1.ps1                                 # Task 1a, 1b and 1c, 3 repetitions each
.\run-task2.ps1 -UsbDir E:\lab1 -CopyFirst   # asks for an eject and replug before each
     run
.\dismount-volume.ps1 -Drive D               # or: empty the drive's cache without
    unplugging it
.\summary.ps1                                # medians and speed-ups in results\summary.
    csv
python ..\report\make_charts.py              # charts in report\figures
```

B Raw data

All measured runs are stored as CSV files in the `lab1/results` folder of the source code. The files `task1a.csv`, `task1b.csv`, `task1c.csv` and `task2.csv` contain every individual run with its time, memory, counts and the result of the correctness check. The file `summary.csv` contains, for every configuration, the median, minimum and maximum time, the speed-up and the median memory. The tables and charts of Task 1 use the three repetitions of 4 October 2026; `make_charts.py` selects them by date, because the CSV files also contain a later set of repetitions. In `task2.csv`, the runs used in this report have the variant `usb-cold`; the single earlier run with the variant `usb` came from the file cache and is not used. The console output of the additional experiments of questions 3 and 6, which were started by hand and are therefore not in these CSV files, is stored in `extra_runs_2026-10-13.txt`.